DEVOPS & AUTOMATION LAB REPORT

K.R. Mangalam University

Submitted By: Sagar
Roll Number: 2301010312
University: K.R. Mangalam University
Faculty Name: Not Available

INDEX

S.No.	Experiment Title
1	Experiment 1 – Infrastructure Automation using Ansible
2	Experiment 2 – Infrastructure Provisioning using Terraform
3	Experiment 3 – Monitoring using Prometheus and Grafana
4	Experiment 4 – Centralized Logging using ELK Stack
5	Experiment 5 – DevSecOps Integration
6	Experiment 6 – Feature Flags, Load Testing and Selenium
7	Viva Questions and Answers
8	Conclusion

EXPERIMENT 1

Aim

To automate application deployment and server configuration using Ansible.

Theory

Ansible is an open-source automation tool used for configuration management, software deployment, and orchestration. It uses YAML-based playbooks to automate repetitive tasks.

Commands & Configuration

SHELL

sudo apt update sudo apt install ansible -y ansible --version

INVENTORY (INI)

[webserver] localhost ansible_connection=local

PLAYBOOK (YAML)

--- - hosts: webserver become: yes tasks: - name: Install Nginx apt: name: nginx state: present

EXECUTE (SHELL)

ansible-playbook -i inventory.ini nginx.yml

Observation

- Ansible successfully automated package installation.
- Nginx was deployed using a playbook.

Result

Infrastructure automation was performed successfully using Ansible.

EXPERIMENT 2

Aim

To provision infrastructure using Terraform.

Theory

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp. It allows users to describe infrastructure using configuration files.

Commands & Configuration

SHELL

terraform version terraform init terraform plan terraform apply

MAIN.TF (TERRAFORM)

resource "local_file" "demo" { filename = "hello.txt" content = "Terraform Working" }

Observation

- Resources were created using code.
- Terraform maintained infrastructure state.

Result

Infrastructure provisioning was successfully implemented using Terraform.

EXPERIMENT 3

Aim

To monitor system resources using Prometheus and Grafana.

Theory

Prometheus collects metrics while Grafana visualizes these metrics through dashboards.

Commands

SHELL

```
docker run -d --name prometheus -p 9090:9090 prom/prometheus docker run -d --name node-exporter -p 9100:9100 prom/node-exporter docker run -d --name grafana -p 3000:3000 grafana/grafana
```

Metrics Monitored

- CPU Usage
- RAM Usage
- Disk Usage
- Network Usage

Observation

Prometheus collected metrics successfully and Grafana displayed visual dashboards.

Result

System monitoring was successfully implemented.

EXPERIMENT 4

Aim

To implement centralized logging using the ELK Stack.

Theory

The ELK Stack consists of:

- **Elasticsearch:** Stores log data.
- **Logstash:** Processes and forwards logs.
- **Kibana:** Visualizes logs.

Commands

ELASTICSEARCH (SHELL)

```
docker run -d \ --name elasticsearch \ -p 9200:9200 \ -e "discovery.type=single-node" \ -e "xpack.security.enabled=false" \ docker.elastic.co/elasticsearch/elasticsearch:8.11.1
```

KIBANA (SHELL)

```
docker run -d \ --name kibana \ -p 5601:5601 \ --link elasticsearch:elasticsearch \ docker.elastic.co/kibana/kibana:8.11.1
```

Observation

Logs were collected and viewed through Kibana dashboards.

Result

Centralized logging was implemented successfully.

EXPERIMENT 5

Aim

To integrate security into the DevOps workflow using DevSecOps practices.

Theory

DevSecOps combines Development, Security, and Operations. Security testing is integrated into CI/CD pipelines.

Jenkins Pipeline & Security Tools

JENKINS PIPELINE (GROOVY)

```
pipeline { agent any stages { stage('Build') { steps { echo 'Building' } } stage('Security Scan') { steps { echo 'Scanning' } } stage('Deploy') { steps { echo 'Deploying' } } } }
```

SECURITY TOOL - OWASP DEPENDENCY CHECK (SHELL)

```
dependency-check.sh --scan .
```

Observation

Vulnerabilities can be detected before deployment.

Result

Security scanning was integrated into the software lifecycle.

EXPERIMENT 6

Aim

To implement feature flags, perform load testing, and automate testing using Selenium.

PART A: Feature Flags

Theory: Feature flags allow enabling or disabling functionality without modifying application code.

PYTHON

```
FEATURE_NEW_UI = True if FEATURE_NEW_UI: print("New UI Enabled") else: print("Old UI Enabled")
```

Result: Feature functionality was successfully controlled using feature flags.

PART B: Load Testing using JMeter

Theory: Load testing evaluates system performance under multiple concurrent users.

INSTALLATION (SHELL)

sudo apt update sudo apt install openjdk-17-jdk -y

Download and Extract JMeter.

Configuration: Threads: 25 | Ramp-Up: 5 seconds | Loop Count: 10

HTTP REQUEST CONFIGURATION

Protocol: https Server: www.google.com

Observation: JMeter generated multiple concurrent requests and provided performance statistics.

Result: Load testing was successfully completed.

PART C: Selenium Automation

Theory: Selenium automates browser actions and web application testing.

INSTALLATION (SHELL)

sudo apt install python3.12-venv -y python3 -m venv venv source venv/bin/activate pip install selenium

SCRIPT (PYTHON)

from selenium import webdriver driver = webdriver.Chrome() driver.get("https://www.google.com") print(driver.title) driver.quit()

Observation: Browser automation was successfully executed.

Result: Automated testing using Selenium was completed successfully.

VIVA QUESTIONS AND ANSWERS

1. What is DevOps?
A methodology that combines development and operations to improve software delivery.

2. What is Ansible?
A configuration management and automation tool.

3. What is Terraform?
An Infrastructure as Code tool.

4. What is Grafana?
A visualization platform for monitoring data.

5. What is Prometheus?
A metrics collection and monitoring system.

6. What is ELK?
Elasticsearch, Logstash, and Kibana.

7. What is DevSecOps?
Security integrated into DevOps practices.

8. What is JMeter?
A load testing tool.

9. What is Selenium?
A browser automation framework.

10. What is a Feature Flag?
A mechanism to enable or disable features without changing code.

CONCLUSION

The DevOps & Automation laboratory experiments were successfully performed. Infrastructure automation was implemented using Ansible, infrastructure provisioning was carried out using Terraform, monitoring was achieved using Prometheus and Grafana, centralized logging was implemented using ELK Stack, DevSecOps practices were integrated into the CI/CD workflow, and automated testing was performed using Selenium. These experiments demonstrated modern DevOps practices used in industry for automation, monitoring, security, and software delivery.